Critical Design Review Document

OpenSpace and Climate Reanalyzer Interface

Team Violet:

Brianna Gannett, Brennon Tiensivu, Hunter Savarese, Matthew Keast, Emily Scott

Clients:

Dr. Sean Birkel, Mr. Shawn Laatsch, Micah Acinapura

Version Control

Version	Date Finalized
1.0	12-08-25
2.0	12-11-25

Table of Contents

1 Introduction **1**
1.1 Purpose of This Document 1
1.2 References 1
2 Problem and Solution **1**
2.1 Problem Statement 1
2.2 Proposed Solution 2
3 Requirements **2**
3.1 Functional Requirements 2
3.2 Nonfunctional Requirements 3
4 Architecture **3**
4.1 Architecture and Tech Stack 3
4.2 User Interface Navigation 4
5 Accessibility and Inclusivity **4**
5.1 Color and Font Choices 4
5.2 Buttons/App Functionality 4
6 DevOps and CI/CD **5**
6.1 Workflows 5
6.2 Testing 5
7 Timeline **6**
7.1 Concluding Semester (Fall 2025) 6
7.2 Upcoming Semester (Spring 2026) 6
Appendix A – System Requirements Specification (SRS) **7**
Appendix B – System Design Document (SDD) **24**
Appendix C – User Interface Design Document (UIDD) **36**

1 Introduction

This project, titled *OpenSpace and Climate Reanalyzer Interface,* is a comprehensive capstone project for University of Maine Computer Science students on V-Model Violets (formerly known as Team Violet). This project will be developed in collaboration with Dr. Sean Birkel, a Research Assistant Professor at the Climate Change Institute, and Sean Laatsch, the Director of the Versant Planetarium at the University of Maine. This project will also be collaborating with Dr. Carter Emmart and Micah Acinapura from the OpenSpace team. The goal of this project is to create an image map server that utilizes data from the Climate Reanalyzer, enabling its use by OpenSpace and in planetariums or other research settings.

1.1 Purpose of This Document

The purpose of this Critical Design Review (CDR) is to review, critique, and reflect upon the entire process behind the OpenSpace and Climate Reanalyzer Interface project thus far. This includes all design, development, documentation, communication, feedback, and other processes that have been undertaken by The V-Model Violets and our clients over the past three months.

1.2 References

Birkel, S. (n.d.). Weather forecasts. Climate Reanalyzer, Climate Change Institute, University of Maine. https://climatereanalyzer.org/

OpenSpace Team. (n.d.). OpenSpace. American Museum of Natural History. https://www.openspaceproject.com

Apache Software Foundation. (n.d.). Apache HTTP Server Project. https://httpd.apache.org/

Sommerville, I. (2015). Software engineering (10th ed.). Pearson Education.

2 Problem and Solution

This section will discuss the problem that our clients wanted us to address and our proposed solution to the problem.

2.1 Problem Statement

The Climate Reanalyzer at the University of Maine collects a vast amount of weather data both globally and for specific regions for the purposes of study and analysis. A lot of this data goes unused or is analyzed just for class purposes at the University of Maine. Dr. Sean

Birkel and Mr. Shawn Laatsch would like it if this data were to be more widely available and utilized.

2.2 Proposed Solution

Team V-Model Violets will, with occasional guidance or assistance from Mr. Acinapura, create the OpenSpace and Climate Reanalyzer Interface. This product will take data collected from the Climate Reanalyzer and translate it into OpenSpace to be presented on a 3-dimensional projection. This way, the Climate Reanalyzer's data can be viewed in planetariums around the globe, including in the planetarium on campus, ensuring the data is not wasted potential.

3 Requirements

This section will briefly discuss the requirements that were laid out for the development of this project. These are discussed in more detail and with diagrams in the SRS document.

3.1 Functional Requirements

For our project, we have a total of 7 functional requirements:

- **Map-Tile Conversion:** This requirement is the highest priority requirement. Our program must be able to convert to a tile-based format for OpenSpace to use successfully.

- **Downloading and Saving Tile Files:** The program should be able to download files from our server to use locally, for instance, where planetariums may not have consistent internet connections.

- **Admins Uploading Large Data Files:** The program must allow for admin users to add large amounts of data to the server.

- **The System Listing Climatereanalyzer Maps:** The program has to have a way for users to browse for maps to display within OpenSpace. Through searching or with a calendar picker.

- **Multiple Tile Levels:** The tiled index of maps must be created in a way that it contains multiple zoom levels for higher resolutions when zoomed into the globe within OpenSpace.

- **Animating Maps and Tile Data:** Our maps need to be created in a way that allows users to create short animated slideshows of weather events.

- **Error and Feedback Handling:** Our project should be clear when reporting errors to users, with instructions on how to resolve them.

3.2 Nonfunctional Requirements

Our Non-functional requirements are split into 6 different categories:

- **Performance:** This category focuses on ensuring our project runs smoothly, maintains 99% uptime, and recovers from errors or maintenance restarts quickly.

- **Platforms:** This category ensures our project works on all major operating systems and on Chrome, our target browser. This also includes making sure our system interacts correctly with our server and with OpenSpace.

- **User Experience:** This category focuses on making sure the user experience is smooth, with an easy-to-understand UI.

- **Error Handling:** This category of requirements is to make sure we have a robust system for error handling. This helps make sure we keep to our other standards of uptime and maintainability.

- **Code Standards:** Our coding standards requirements are to keep our developers consistent and make sure our code is easy to read and understand. This is important for our project as we are open source and our code is publicly available on GitHub.

4 Architecture

This section is dedicated to discussing the architecture of our project. In this section, the frameworks, languages, and overall structure of the project will be discussed in detail.

4.1 Architecture and Tech Stack

The underlying architecture for our project is a little unconventional compared to most applications. Rather than having one big application that involves a frontend and a backend, this project will be composed of almost two smaller application projects that use each other to work with OpenSpace. Our project will consist of an image tile server that will have large

images translated with GDAL that can be accessed via URL. This server will be the access point for our User Interface that will use OpenSpace imports to allow it to be used within the OpenSpace application.

4.2 User Interface Navigation

The Navigation Diagram shown in Figure 3.1 shows the two page types that will be implemented in our user interface and how the user will navigate between them. Upon opening the Climate Reanalyzer x OpenSpace Interface within the OpenSpace application, the user will be directed to the Landing Page. From there, the user will see multiple buttons that each lead to a corresponding Map Search Page. The user may click the back button on the top left to return to the Landing Page.

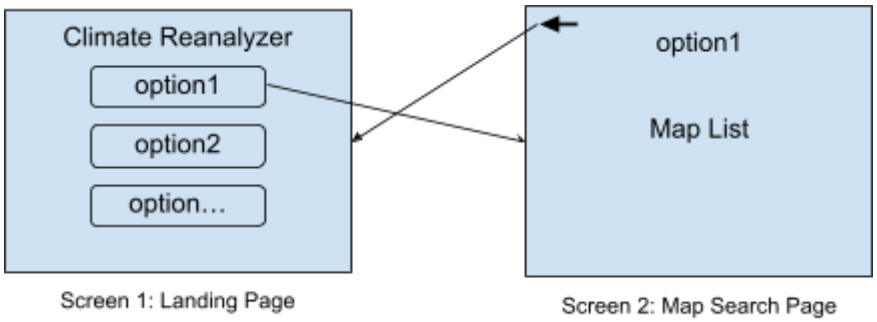

Figure 1: Navigation Diagram for Climate Reanalyzer x OpenSpace Interface

5 Accessibility and Inclusivity

This section is devoted to the steps we took to make our product more accessible and inclusive to a wider customer base.

5.1 Color and Font Choices

Across the entirety of the User Interface for the project, we made sure to limit the colors used to five colors. These five colors range from black to white along the blue spectrum. We, V-Model Violets, made sure to use them in a way that curated high contrast and readability for our users. Keeping all colors on the blue spectrum allowed us to ensure that the colors would not appear too similar if a person were to have nearly any form of color-blindness.

For fonts, we made sure to use a Serif font as they have an easier time being read than other font families. This way, the font would be legible for the largest possible number of users while still fitting in with the look of OpenSpace's already established User Interface.

5.2 Buttons/App Functionality

In order to make our buttons more user-friendly and accessible, we decided to outline our buttons in white to help them stand out against the dark background, as well as change the fill color in the background to be lighter than the background to make it clear that it is interactable. We also included a function that, when you hover over a button, the button will shade. This makes it even clearer that the button is interactable.

6 DevOps and CI/CD

This section is devoted to discussing the DevOps strategies we used for continuous integration. In this section, there will be some discussion about the testing procedures we plan to implement in the future.

6.1 Workflows

Our CI/CD setup keeps things consistent and catches issues before they become problems. We use a commit message validator to make sure everyone follows the same format. It sounds picky, but it actually makes a huge difference when you're trying to track down when something changed or why. All pull requests go through code review too, which gives the team a chance to catch bugs early, share what they're learning, and make sure we're all maintaining the same standards. These aren't complicated processes, but they give us a solid base to build on as the project gets bigger. Along with these workflows, we have also implemented linting and style checking using Husky. This will help keep code clean and well-structured for readability and consistency.

6.2 Testing

As the project grows, we're planning to add more testing to make sure everything works reliably on both the frontend and backend. We'll be writing React unit tests to verify that components behave correctly and catch UI bugs before they reach users. We also want to set up automated checks that test whether each image set on the tile server is actually accessible at its URL. This will help us catch broken links or missing resources before they make it to production. Adding these tests will make the system more reliable and keep our CI/CD pipeline solid as we continue building out the codebase.

7 Timeline

In this section, we will discuss the current idea of our timeline for the concluding and upcoming semester.

Figure 2: Year-long Project Timeline

7.1 Concluding Semester (Fall 2025)

In the past few months, comprising the Fall semester of 2025, we planned to finish most of the initial documentation. We completed the System Requirements Specification, System Design Document, and the User Interface Design Document. These documents defined and outlined the requirements and basic architecture of the Climate Reanalyzer and OpenSpace Interface. Based on these documents, we were able to begin development on the Interface that will link the Climate Reanalyzer and OpenSpace together. We are closing out the semester with a prototype that will function off of a dummy map to give an idea of how the final product will operate.

7.2 Upcoming Semester (Spring 2026)

In the upcoming spring semester, we plan to complete development of the interface that will communicate with a remote map server containing Climate Reanalyzer maps and integrate them into OpenSpace for display. We aim to gain access to the tile server from our client, Dr. Birkel, by early January, and we aim to have enough of the backend work done to be able to project data into the dome by mid February. We also plan to do integration and acceptance testing to make sure the program will be able to function properly and will be intuitive enough for the average OpenSpace user later on in the spring semester.

Should time allow, there are a few features that have been discussed between V-Model Violets and our clients as stretch goals. Those include, but are not limited to, features such as the smooth “animation” of the Climate Reanalyzer data sets over time and the operation of the interface offline.

Appendix A – System Requirements Specification (SRS)

Table of Contents

Appendix A – System Requirements Specification (SRS) **7**
1. Introduction 8
    1.1 Purpose of This Document 8
    1.2. References 8
    1.3. Purpose of the Product 9
    1.4. Product Scope 9
2. Functional Requirements 10
    Map Tile Conversion 11
    Downloading and Saving Map Files 12
    Admins Uploading Large Data Files 14
    System Listing Available Climate Reanalyzer Maps 15
    Multiple Tile Levels 16
    Animating Maps and Data 17
    Error and Feedback Handling 19
3. Non-Functional Requirements 20
4. User Interface 22
5. Deliverables 23
6. Open Issues 23

1. Introduction

This project, titled *OpenSpace and Climate Reanalyzer Interface,* is a comprehensive capstone project for University of Maine Computer Science students on V-Model Violets (formerly known as Team Violet). This project will be developed in collaboration with Dr. Sean Birkel, a Research Assistant Professor at the Climate Change Institute, and Sean Laatsch, the Director of the Versant Planetarium at the University of Maine. This project will also be collaborating with Dr. Carter Emmart and Micah Acinapura from the OpenSpace team. The goal of this project is to create an image map server that utilizes data from the Climate Reanalyzer, enabling its use by OpenSpace and in planetariums or other research settings.

1.1 Purpose of This Document

The purpose of this System Requirements Specification (SRS) document is to define and describe the requirements (functional and non-functional) for the OpenSpace and Climate Reanalyzer Interface. This document will serve as a reference for our team, the clients, and the teaching staff for the Capstone class, outlining the system's structure, design rationale, and technical specifications that support performance, security, and maintainability. The SDD is intended for:

- Capstone Team Members: To lay out the project requirements to ensure the client's needs are met
- Clients: To ensure that the project is on the right track
- University of Maine Faculty: To assess the progress of the project

1.2. References

The table below lists the sources being used for this project currently:

Title	Author(s)	Publisher / Organization	Date	Citation
Climate Reanalyzer	Dr. Sean Birkel	Climate Change Institute, University of Maine	Ongoing	Birkel, S. (n.d.). Weather forecasts. Climate Reanalyzer. https://climatereanalyzer.org/
OpenSpace	OpenSpace Team	American Museum of Natural History	Ongoing	OpenSpace. (n.d.). OpenSpace. https://www.openspaceproject.com

Apache HTTPD Server Documentation	Apache Software Foundation	Open Source	1997-Current	Apache Software Foundation. (n.d.). Apache HTTP Server Project. https://httpd.apache.org/
Software Engineering (10th Edition)	Ian Sommerville	Pearson Education	2015	Sommerville, I. (2015). Software engineering (10th ed.). Pearson Education.

1.3. Purpose of the Product

The Climate Reanalyzer OpenSpace interface project is set out to make the data gathered and used on the University of Maine's Climate Reanalyzer available for use in the OpenSpace planetarium software. This allows for coverage of data that OpenSpace may not have readily available yet. The incorporation of the Climate Reanalyzer into the OpenSpace system will allow for immersive and interactive displays of historical climate data across the globe in a planetarium setting. This product will allow for data such as temperature, climate change data, atmospheric pressure, and much more to be overlaid onto a model of Earth that OpenSpace has integrated into their software. The product will address both missing data and inconsistencies in existing data to provide a more accurate model of Earth in its current state and in previous generations. In conclusion, the product will help enhance science education for teaching and learning.

1.4. Product Scope

The Climate Reanalyzer (CR) system will provide a connection between the Climate Reanalyzer platform and the OpenSpace visualization software that is used in planetariums. Our primary goal is to make global climate and weather data from the CR visual on display for planetariums and other large digital environments.

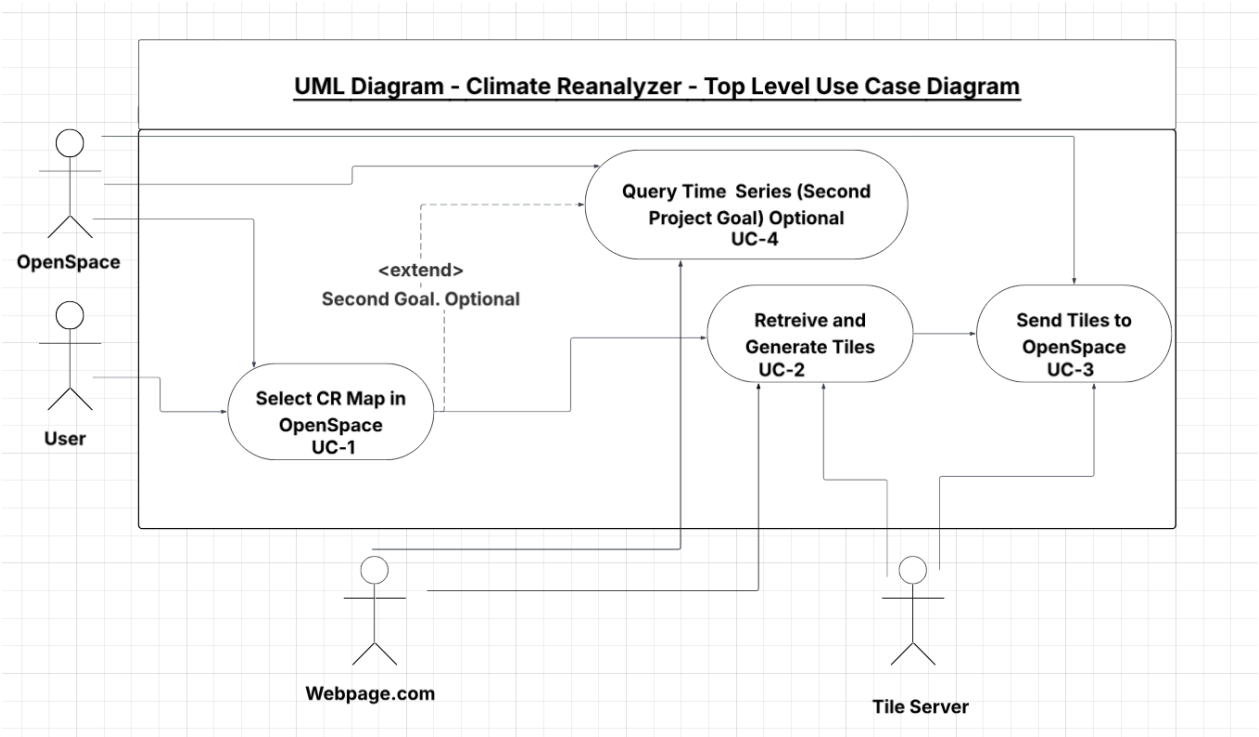

Figure 1.1 Top-Level Use Case Diagram: Climate Reanalyzer

Figure 1.1 shows the overall scope of the system and how it will connect to the actors. The Interface will allow users to select and load climate maps from the webpage. The system will get that information, convert the map data into a tile map that can be sent to OpenSpace for viewing. The optional feature UC-4 will let the system show time series data from the CR, which is the second project goal. This diagram shows the inside system boundary and what the external.

2. Functional Requirements

This section is where the Functional Requirements for the OpenSpace and Climate Reanalyzer Interface are found. These are the requirements that explain what the project will do, and they focus more on specific actions that can be taken and how they will function. There are tests included for each requirement that describe specifically how we will verify the system's function.

Use Case #1: Map-Tile Conversion

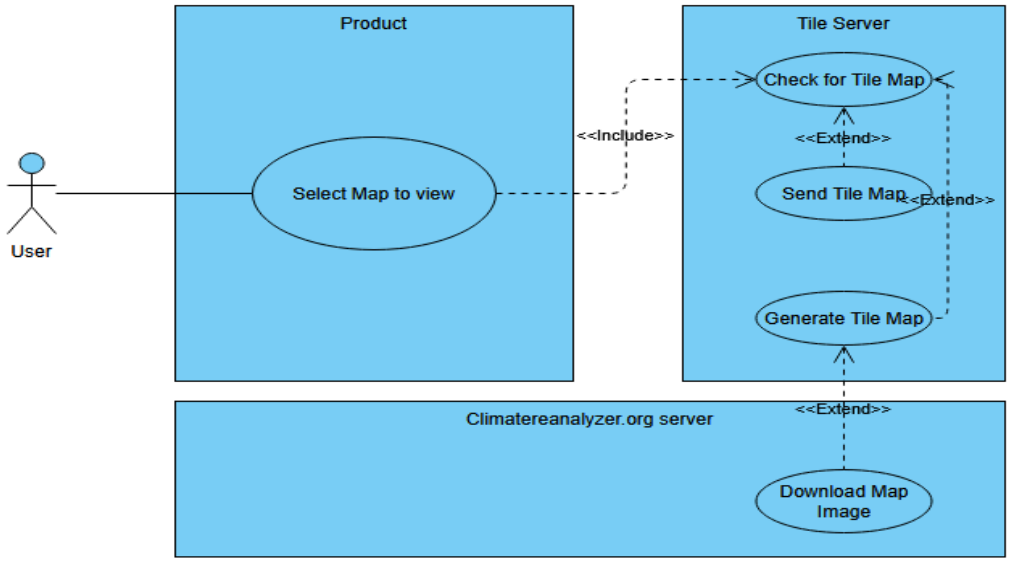

Figure 2.1: Map-tile Conversion Use Case Diagram

Number	1	
Name	Map - Tile Conversion	
Summary	The system shall allow OpenSpace to display a tile map generated from a 2x1 map image when requested.	
Priority	5	
Preconditions	A user must request a map image	
Postconditions	The map image will be displayed within OpenSpace.	
Primary Actor	User	
Secondary Actors	Climatereanalyzer.org image data, Tile Server, OpenSpace, system	
Trigger	When the user selects a map to load.	
Main Scenario	**Step**	**Action**
	1	The user requests a new map.
	2	The System checks if that map is already available.
	3	The system sends requested tile data to OpenSpace for display.
Extensions	**Step**	**Branching Action**
	2a	If not already available, the system downloads the map image.
	2b	The system calls for GDAL and Apache to generate a tile map.
	2c	The system stores the tile map on the tile server.
Open Issues	Should the System automatically download and store all data from Climate Reanalyzer? Should the system only download new maps to the server if the map is requested directly?	

Test for Functional Requirement #1

To test this requirement, we will write some test code that will simulate a call from a user for a specific map. If the map is sent correctly, or if it is downloaded and stored correctly on the server, the test will pass. Otherwise, the test will fail.

Use Case #2: Downloading and Saving Map Files

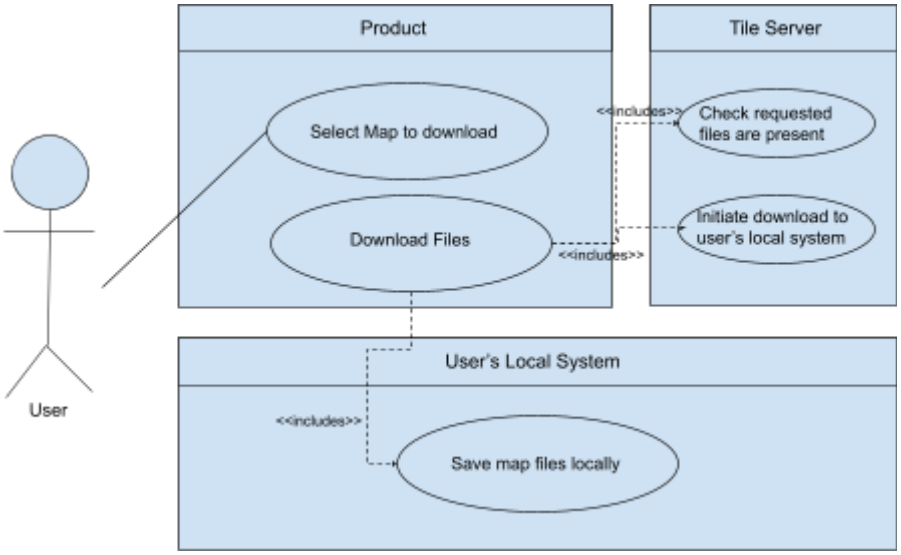

Figure 2.2: Downloading and Saving Tile Files Use Case Diagram

Number	2
Name	Download and Save Map Files
Summary	The system should be able to download and save files containing 2:1 maps, allowing them to be used in the OpenSpace simulation on demand.
Priority	2
Preconditions	● The system has a connection to the internet. ● There are files present in the tile server ● There is sufficient space on the user's system for the download
Postconditions	● The selected maps are successfully saved to local storage. ● The system can access and load the saved maps on demand without needing to redownload them.
Primary Actor	User
Secondary Actors	Tile server, OpenSpace system
Trigger	When the user initiates the request to download maps to use them offline.

Main Scenario	Step	Action
	1	The user selects the tiles to download.
	2	The System connects to the tile server.
	3	The system downloads the files from the tile server and saves them locally to the user's device.
Extensions	**Step**	**Branching Action**
	2a	If the download fails due to the connection to the tile server, the system will alert the user
	3a	If the download fails (due to connection loss), the system pauses and will alert the user.
	3a	If the download fails due to storage issues, the system alerts the user to the insufficient space and prompts them to clear space or choose a different storage location.
Open Issues	If the download fails, should the system automatically attempt to resume the download once the connection has been restored, or should it wait for another prompt from the user?	

Test for Functional Requirement #2

To test this requirement, a request will be sent to the system to download a specific set of tiles for later use in the OpenSpace simulation. Once the request is made, the system should connect to the remote server, retrieve the requested tile files, and store them in the designated local directory. After the download process completes, the storage location will be checked to verify that the tile files exist and are accessible. If the files exist and are accessible, the test passes; otherwise, the test fails.

Use Case #3: Admins Uploading Large Data Files

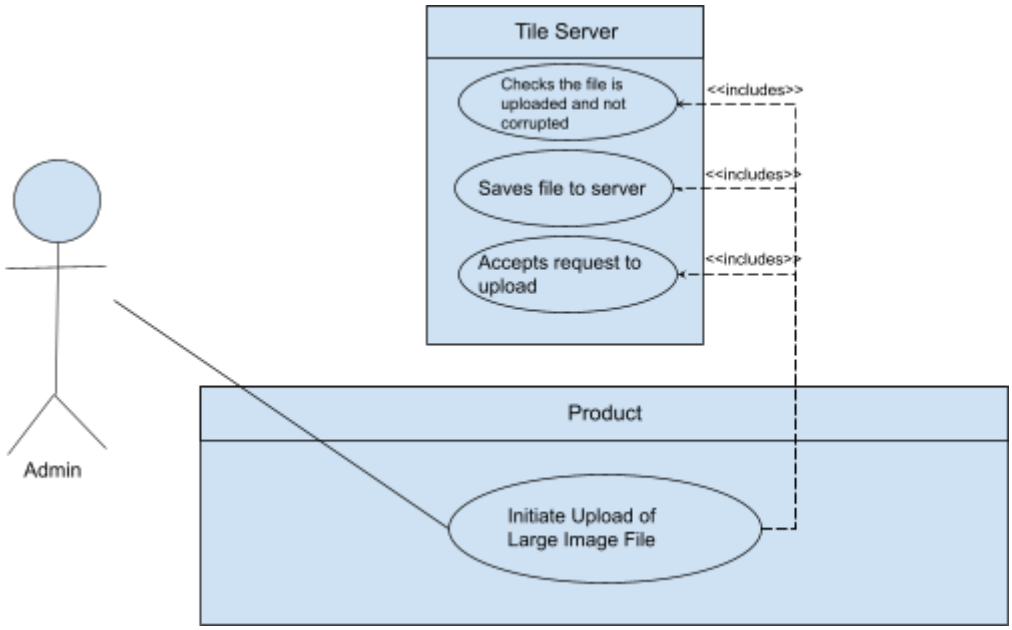

Figure 2.3: Admin Uploading Large Data Files Use Case Diagram

Number	3	
Name	Admins Uploading Large Data Files	
Summary	The system shall allow administrators to upload large source images to be used by the tile server.	
Priority	4	
Preconditions	● The system has a connection to the internet. ● There are files present on the local machine ● There is sufficient space on the server to upload	
Postconditions	● The selected tiles are successfully saved on the server. ● The system can access and load the uploaded image.	
Primary Actor	Admin	
Secondary Actors	Tile server	
Trigger	When the user initiates the request to download tiles to use them offline.	
Main Scenario	**Step**	**Action**
	1	The admin finds a large image file to upload.
	2	The admin connects to the server.
	3	The admin uploads the file to the server
Extensions	**Step**	**Branching Action**
	2a	If the upload fails due to the connection to the tile server, the system will alert the user.
	3a	If the upload fails (due to connection loss), the system pauses and will alert the user.
	3a	If the upload fails due to storage issues, the server alerts the admin to the insufficient space.
Open Issues	If the upload fails, should the server save the upload progress, or should it delete progress until there is sufficient space?	

Test for Functional Requirement #3

To test this requirement, we will attempt to upload an image to the server. Then the server will be checked to see if the images are present. If they are, the test passes. If the images are not present, the test fails.

Use Case #4: The System listing available Climate Reanalyzer Maps

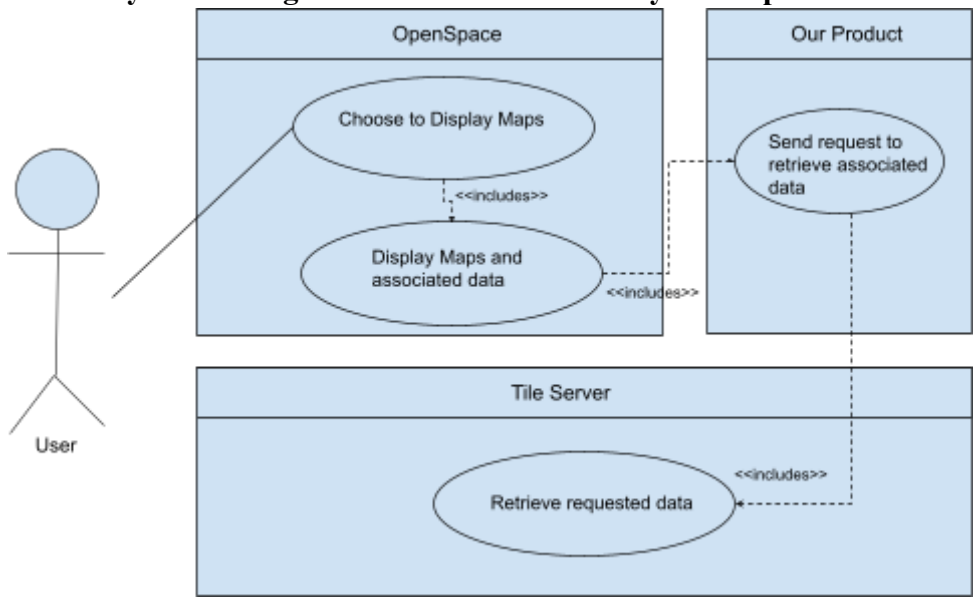

Figure 2.4: Listing Available Climate Reanalyzer Maps Use Case Diagram

Number	4	
Name	List Available CR Maps	
Summary	The system shall expose a list of available Climate Reanalyzer maps with the layer name, date/time, and resolution. So the presenters can pick one in OpenSpace.	
Priority	4	
Preconditions	● The tile server is reachable ● CR list is available	
Postconditions	● The map layer is selected ● Metadata is cached for retrieval.	
Primary Actor	User	
Secondary Actors	Tile server, OpenSpace, ClimateReanalyzer.org	
Trigger	The user opens the “Select map”.	
Main Scenario	**Step**	**Action**
	1	User requests a list of maps.
	2	System fetches map catalog.
	3	System returns a list, user selects one.
Extensions	**Step**	**Branching Action**
	2a	If CR is not reachable. The system will serve the last known local catalog.
Open Issues	Is this possible? Can it auto-refresh?	

Test for Functional Requirement #4

To test this requirement, we will create a mock CR and serve it a catalog. We will then confirm that the program lists accurate data and selection returns the correct layer ID. If it does so, the test passes; otherwise, the test fails.

Use Case #5: Multiple tile levels

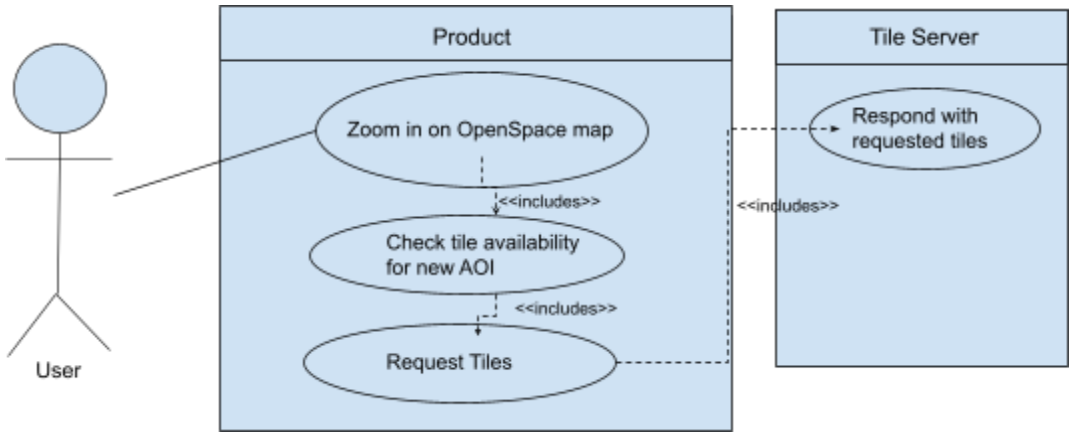

Fig 2.5: Being able to view tile maps on multiple levels, use case diagram

Number	5	
Name	Multiple tile levels	
Summary	The system shall provide inner (zoomed) layers of tiles upon request	
Priority	4	
Preconditions	● The server must successfully connect with OpenSpace ● The map must be loaded in OpenSpace ● The user must zoom in to trigger new tile requests	
Postconditions	● Tile layers will be provided while there is sufficient resolution in the CR image.	
Primary Actor	User	
Secondary Actors	Tile server, OpenSpace, Climatereanalyzer.org	
Trigger	When the user interacts with the OpenSpace interface and zooms in	
Main Scenario	**Step**	**Action**
	1	The user zooms in on the map in OpenSpace.
	2	OpenSpace requests the necessary tiles from the tile server
	3	The tile server responds with the requested image tiles
Extensions	**Step**	**Branching Action**
	2a	If there are more tile levels that can be loaded, OpenSpace requests these from the tile server.

	2b	If no further tile levels remain, OpenSpace will not load zoomed data
Open Issues	Should the system store all of the tile levels individually, or calculate image cropping and transformation upon request for a new tile level?	

Test for Functional Requirement #5

To test this requirement, we will visit server URLs specifying the inner tile layers that should be available and verify that an image is returned. This should call the server and request a tile, which the server should recognize and identify as an available layer, with which it will respond. If the image is displayed at the URL, the test passes; otherwise, the test fails.

Use Case #6: Animating Maps and Data

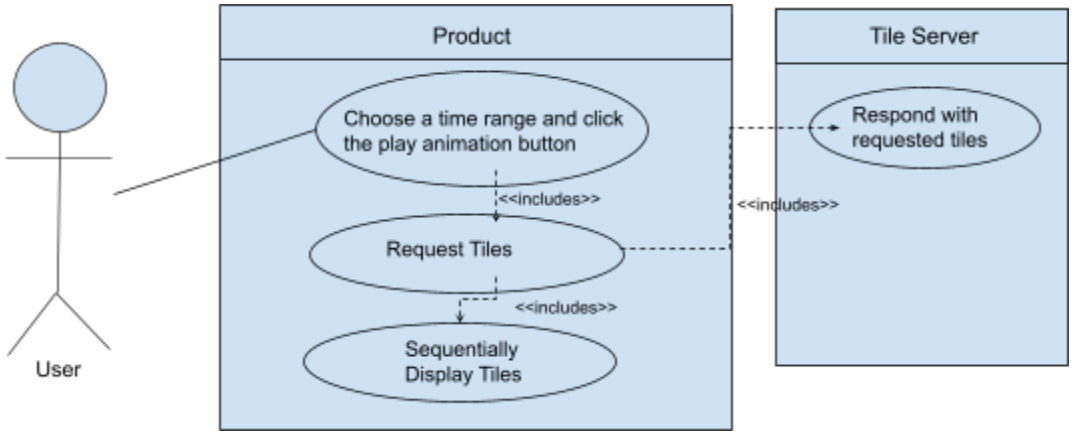

Fig 2.6: Being able to view a certain time span in OpenSpace in a way that looks animated use case diagram

Number	6
Name	Animating Maps and Data
Summary	The system shall display an 'animation' of data over a chosen time range for a region upon request.
Priority	2
Preconditions	● The server must successfully connect with OpenSpace ● The maps must be loaded in OpenSpace
Postconditions	● The maps over the time the user chose will be shown sequentially, giving the idea that an animation for the weather during that time is being shown.
Primary Actor	User
Secondary Actors	Tile server, OpenSpace, Climatereanalyzer.org

Trigger	When the user chooses a time range and chooses to view the 'animation' of the data over that time range.	
Main Scenario	**Step**	**Action**
	1	The user chooses to play the animation.
	2	OpenSpace requests the tiles for those times from the tile server.
	3	The tile maps are shown sequentially to give the illusion of an animation
Extensions	**Step**	**Branching Action**
	2a	If there are no tiles present in the tile server for the time the user requested, the system will display an error message explaining that the time range is invalid.
Open Issues	Should users be allowed to adjust playback speed?	

Test for Functional Requirement #6

> A request will be sent for a supported time-series dataset over a short range of timestamps. The system should retrieve and display tiles in order. If the animation shows correct visual progression without failure, the test passes; otherwise, it fails. Request a tile, which the server should recognize and identify as an available layer, with which it will respond. If the image is displayed at the URL, the test passes; otherwise, the test fails.

Use Case #7: Error and Feedback Handling

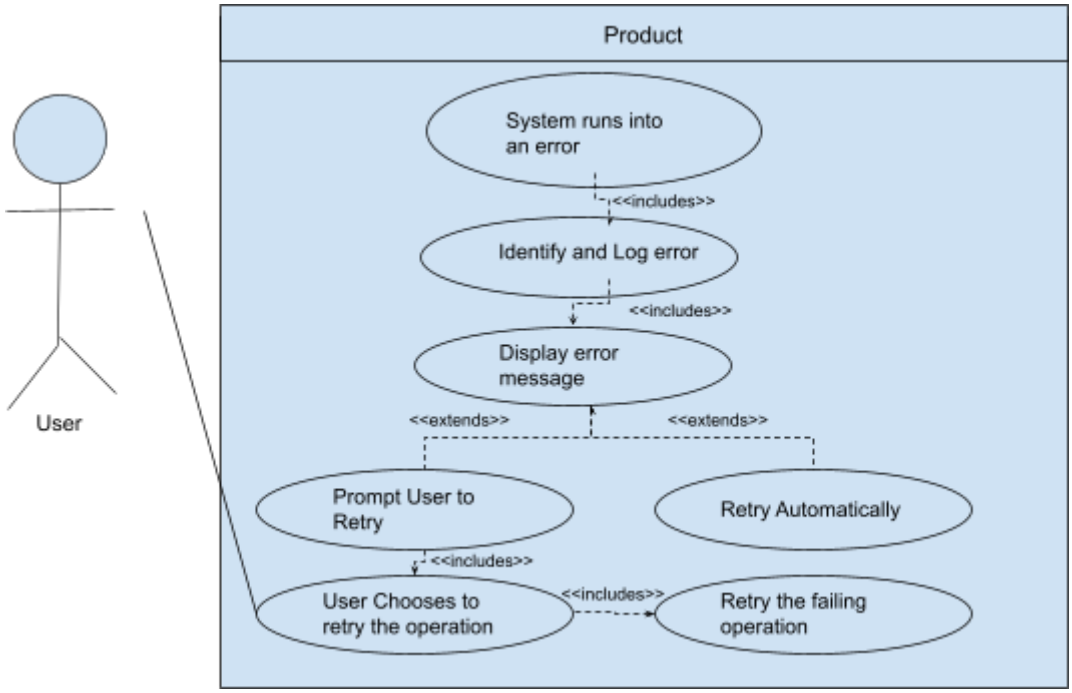

Fig 2.7: Being able to view a certain time span in OpenSpace in a way that looks animated use case diagram

Number	7	
Name	Handling Errors and Status Feedback	
Summary	The system shall alert users when tile retrieval, downloads, uploads, or catalog refreshes fail, displaying descriptive and actionable error messages.	
Priority	4	
Preconditions	● The system encounters an issue (e.g., connection failure, missing tiles, invalid user request)	
Postconditions	● The user is shown an error message that describes the error and why it happened.	
Primary Actor	User	
Secondary Actors	Tile server, OpenSpace	
Trigger	An error or unexpected behavior happens while the program is running	
Main Scenario	**Step**	**Action**
	1	A request or operation fails.
	2	The system identifies the error and its cause.
	3	The system displays an informational message to the user
Extensions	**Step**	**Branching Action**
	3a	The system logs the error and relevant logistical information, such as time, for the admin to analyze later.
	3b	If retrying the action is an option, the system prompts the user to retry.
Open Issues	Are there certain features that the system should automatically restart without prompting the user?	

Test for Functional Requirement #7

A failure will be forced by removing server connectivity and initiating a tile request. If the user receives a descriptive error message that matches the expected failure type, the test passes; otherwise, it fails.

3. Non-Functional Requirements

This section is where the Nonfunctional Requirements for the OpenSpace and Climate Reanalyzer Interface are found. These are the requirements that explain how the product will operate, and they focus more on how the project operates and less so on what the project can do. There are tests included for each requirement that describe specifically how we will verify the system's function.

Non-Functional Requirement #1

Requirement ID. 3.1:
The application should maintain stable performance under simulated concurrent usage, with response times under 2 seconds for all core functionalities when tested with a representative load using automated tools.
Priority - 3
Test: Run a stress test of many uses at once over a short period of time. If everything works as planned, it will pass; otherwise fail.

Non-Functional Requirement #2

Requirement ID. 3.2:
The application should be able to run on Chrome and all major Operating Systems.
Priority - 3
Test: Attempt to open the browser application on Chrome, as well as on Windows, Mac, and Linux systems. If all run successfully it will pass; otherwise, it fails.

Non-Functional Requirement #3

Requirement ID. 3.3:
The application's interface should be intuitive enough that new users are able to complete core application functions (e.g., downloading tiles, uploading maps) without direction at least 85% of the time.
Priority - 5
Test: Perform usability testing with a handful of various people and gauge their understanding of the program's operation through observation and post-test questions. If the vast majority of people (at least 85%) can understand with little assistance from the proctor, the program

passes; otherwise, it fails.

Non-Functional Requirement #4

Requirement ID. 3.4:
The application should be able to run both with and without an internet connection.
Priority - 2
Test: Test that the base functions of the application work both when connected to wifi and with no wifi or service connection.

Non-Functional Requirement #5

Requirement ID. 3.5:
The application must detect corrupted or missing map/tile data and notify the user with a clear error message, without crashing or losing existing unsaved work.
Priority - 3
Test: Provide corrupted or incomplete data files and verify that the application displays a descriptive error message and continues functioning normally.

Non-Functional Requirement #6

Requirement ID. 3.6:
The application's codebase shall conform to the group's coding standards and be structured to facilitate maintainability
Priority - 1
Test: Read through all code to determine whether or not the code follows the standards set by our group.

Non-Functional Requirement #7

Requirement ID. 3.7:
The application should work with the target OpenSpace versions and run on the ARCSIM-provided Linux VM.
Priority - 4
Test: Install ARMSIM VM, validate it works with OpenSpace.

Non-Functional Requirement #8

Requirement ID. 3.8:
The tile service must be ready within 20 seconds after the process starts on the ARCSIM VM.
Priority - 3
Test: Restart the service, measure the time it takes to start.

Non-Functional Requirement #9

Requirement ID. 3.9:
All source code shall be publicly accessible on GitHub
Priority - 3
Test: Verify that any GitHub user can access the repository holding the source code

Non-Functional Requirement #10

Requirement ID. 3.10:
The application shall be available 99% of the time
Priority - 4
Test: Log tile server uptime for a week through requests at regular intervals and calculate if availability is 99% or better

4. User Interface

See User Interface Design Document for *OpenSpace and Climate Reanalyzer Interface*

5. Deliverables

The following deliverable items will be supplied as described below to the clients, Dr. Sean Birkel and Shawn Laastch, throughout the development of the specified software. Most deliverable items will be made available as both a hard copy and a digital PDF.

Item	Format	Date of Delivery
Systems Requirement Specification	-Document Hard Copy -Document Digital Copy	10/29/2025
System Design Document	-Document Hard Copy -Document Digital Copy	11/17/2025
User Interface Design Document	-Document Hard Copy -Document Digital Copy	12/3/2025
User Manual	-Document Hard Copy -Document Digital Copy	5/8/2026
Administrator Manual	-Document Hard Copy -Document Digital Copy	5/8/2026
Copies of all Biweekly Status Reports	-Document Hard Copy -Document Digital Copy	Every two weeks-Mondays starting 10/20/2025
All source code	-GitHub Repository	5/8/2026
The executable program	-Web Application -Open Space Interface	5/8/2026

6. Open Issues

The following are issues that have been identified and not yet concluded. These issues will be addressed later in the development process according to the projected resolution dates.

- Tile server availability (2/16/2026)
- Function to process images from climate reanalyzer into tiles (2/2/2026)
- Function to select desired tiles based on area of interest and available data (2/9/2026)
- WMS file to store data for each map to be loaded into OpenSpace (2/23/2026)
- OpenSpace asset files for each map to be loaded (2/23/2026)
- JS webpage that can be opened within OpenSpace to browse the CR catalog (3/30/2026)

Appendix B – System Design Document (SDD)

Table of Contents

Appendix B – System Design Document (SDD) 24

1 Introduction 25

1.1 Purpose of This Document 25

1.2 References 25

2 System Architecture 26

2.1 Architectural Design 26

2.2 Decomposition Description 27

3 Persistent Data Design 28

3.1 Database Descriptions 28

3.2 File Description 29

Directory Summary 29

File Descriptions 31

4 Requirements Matrix 34

1 Introduction

This project, titled *OpenSpace and Climate Reanalyzer Interface,* is a comprehensive capstone project for University of Maine Computer Science students on V-Model Violets (formerly known as Team Violet). This project will be developed in collaboration with Dr. Sean Birkel, a Research Assistant Professor at the Climate Change Institute, and Sean Laatsch, the Director of the Versant Planetarium at the University of Maine. This project will also be collaborating with Dr. Carter Emmart and Micah Acinapura from the OpenSpace team. The goal of this project is to create an image map server that utilizes data from the Climate Reanalyzer, enabling its use by OpenSpace and in planetariums or other research settings.

1.1 Purpose of This Document

The purpose of this System Design Document (SDD) is to detail the architecture, components, interfaces, and data flow for the OpenSpace and Climate Reanalyzer Interface. This document will serve as a blueprint for our team, the clients, and the teaching staff for the Capstone class, outlining the functionality of the system, possible constraints, and expectations regarding performance, security, and design. The SDD is intended for:

- Capstone Team Members: To guide the group through the implementation process, as well as ensure the client's needs are met.
- Clients: To show how we plan to construct the system in a transparent and informative way
- University of Maine Faculty: To assess the progress of the project

1.2 References

The table below lists the sources being used for this project currently:

Title	Author(s)	Publisher / Organization	Date	Citation
Climate Reanalyzer	Dr. Sean Birkel	Climate Change Institute, University of Maine	Ongoing	Birkel, S. (n.d.). Weather forecasts. Climate Reanalyzer. https://climatereanalyzer.org/
OpenSpace	OpenSpace Team	American Museum of Natural History	Ongoing	OpenSpace. (n.d.). OpenSpace. https://www.openspaceproject.com
Apache	Apache Software	Open Source	1997-Curre	Apache Software

HTTPD Server Documentation	Foundation		nt	Foundation. (n.d.). Apache HTTP Server Project. https://httpd.apache.org/
Software Engineering (10th Edition)	Ian Sommerville	Pearson Education	2015	Sommerville, I. (2015). Software engineering (10th ed.). Pearson Education.

2 System Architecture

This section introduces the general architecture of the OpenSpace and Climate Reanalyzer Interface. Section 2.1 features the general architectural design (including a diagram and a brief write-up). Section 2.2 features the more detailed decomposition design. It has a diagram that is a more broken-down and specific version of the one found in 2.1, as well as a brief write-up.

2.1 Architectural Design

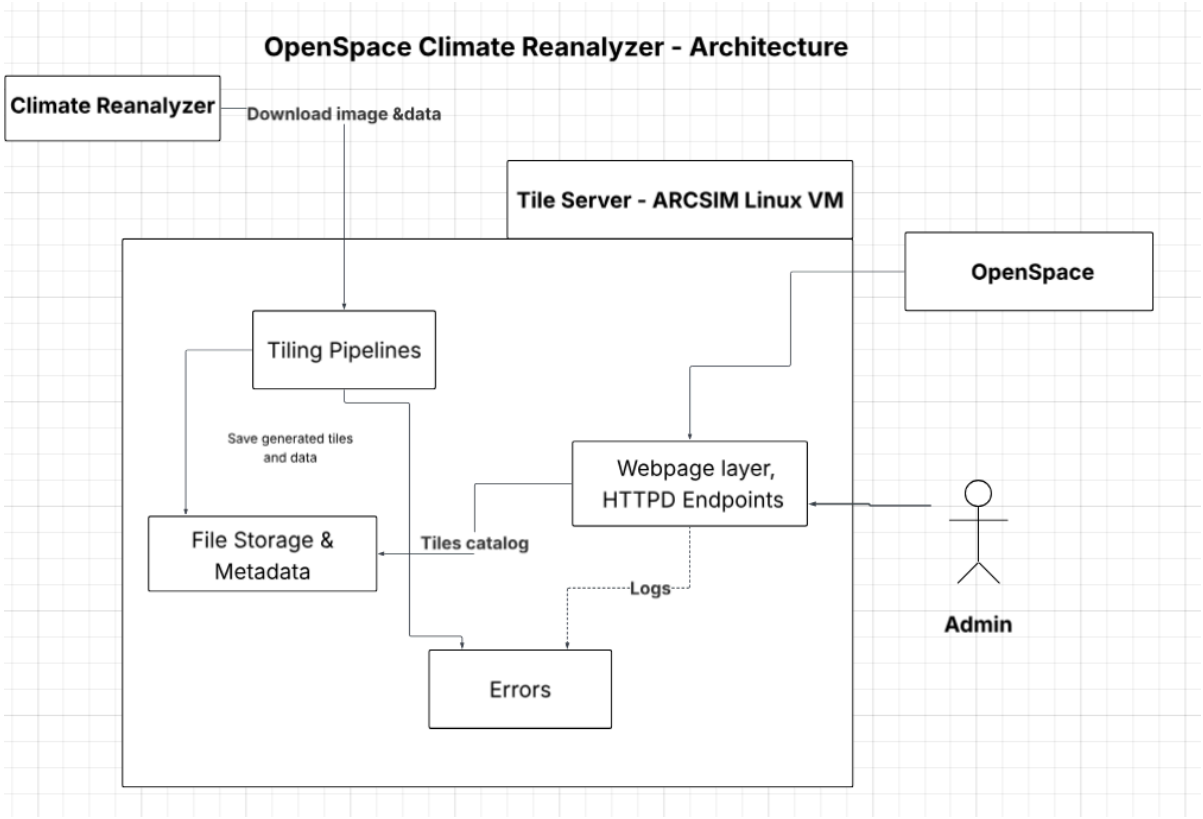

Figure 2.1 OpenSpace - Climate Reanalyzer Architecture

The OpenSpace and Climate Reanalyzer system is a client-server architecture that connects available data from the Climate Reanalyzer to the OpenSpace platform. Figure 2.1 shows the architecture and how the components interact. The goal is to show data flow between the tile server and OpenSpace's platform. This design will allow OpenSpace to request the map tiles it needs.

At the top, the Climate Reanalyzer (CR) is an external data provider. These images will be fetched by the tile server, which will convert the map images to smaller tiles using GDAL utilities. The Tile Server will expose the API via Apache HTTPD that lets OpenSpace request individual tiles or map catalogs. Administrators can also upload new images through the same API. When OpenSpace requests a new map, the server will check whether it already exists. If not, it will download the source image and generate the required tiles, then serve them to the client.

The system runs on a Linux virtual machine (ARCSIM VM) provided by the University of Maine. It will use Apache HTTPD as the web service and GDAL for the image tiling and JSON metadata handling. OpenSpace will communicate with the server via HTTPS requests to retrieve tiles and catalogs for visualization in the planetarium environment. This setup will allow efficient performance and cross-platform compatibility.

2.2 Decomposition Description

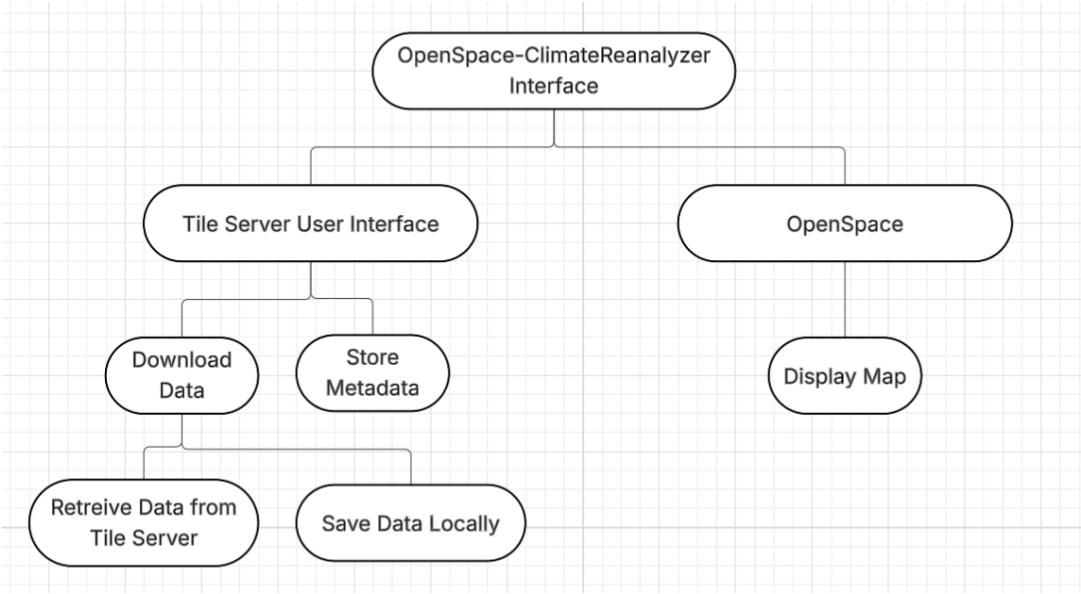

Figure 2.2 OpenSpace - Climate Reanalyzer Decomposition Diagram

This section describes the decomposition of the OpenSpace and Climate Reanalyzer Interface system we are developing. It highlights and describes a more in-depth description of what was described in section 2.1. Our system is mostly procedural, hence the structural decomposition diagram seen above (Figure 2.2).

Our system is a client-server model, which means that clients request services and servers provide them. For the OpenSpace and Climate Reanalyzer Interface, the client would be the user

interface for the Tile Server that is brought up in the OpenSpace application itself. The server is the actual tile server, which holds the maps and metadata that have been previously uploaded.

Our system's functions are broken down into two main sections: actions handled through the tile server and actions handled through OpenSpace itself. The main functions of the Tile Server are to download data to the user's local system and to store the associated metadata. The function that OpenSpace serves is to display the maps that have been saved or cached from the tile server.

3 Persistent Data Design

In this section, we will cover Database design and File Structure. The design of the database will show how we will store the data in our server and how it interacts with the interface and Openspace. Our file structure shows the direct format of the server along with datatypes. This information will be used to build our product, and can work as a guide for developers.

3.1 Database Descriptions

For our project, we need to create a Tile server that will serve as a database of maps and tiled map data. Our product will be interfacing this data with the Openspace Project to allow Openspace to load and display these maps when requested. The database should not be complex and should not require any form of login authentication, as all of the maps are publicly sourced and free to access by anyone. Access to the database will be exclusively through a browser within Openspace, and will only allow for downloading of files and requesting new maps to be added to the database from ClimateReanalyzer. Manually adding and deleting files will only be allowed through direct access to the Linux machine provided to us by Dr. Sean Birkel.

Data types in the database itself include:

- Map_id: uuid, <1Kb, The specific ID for each map.
- Rectangular_map: PNG image, 10-100Mb, The unaltered base image of the map
- Map_tile: png image, 1-10Mb, A single Tile of the map after modification
- tiled_map_file, tif compressed image, 1-10Gb, The tiled tif format compressed image
- Tiled_map_data: WMS tile image index file, 10Mb, Index file for getting each tile.
- Map_metadata: JSON, <100Kb, JSON data used to communicate with the server.

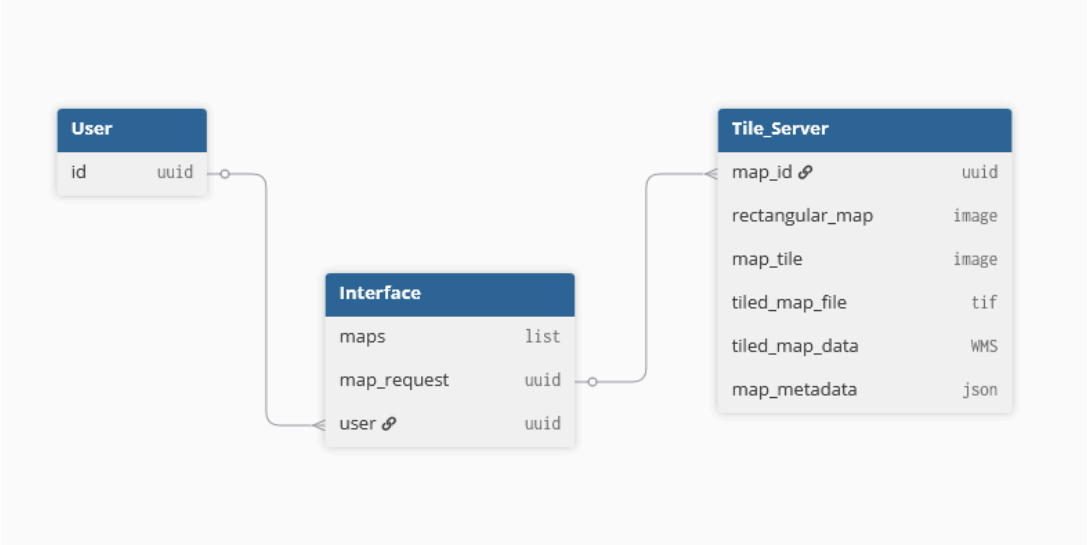

Figure 3.1 OpenSpace - Climate Reanalyzer Database Schema

3.2 File Description

File Structure:

```
/OpenSpace-x-ClimateReanalyzer/
├─ .devcontainer/
│  └─ devcontainer.json
├─ .github/
│    └─ CODEOWNERS
│    └─ workflows/
│        └─ validate-commit-message.yml
├─ webconf/
│  └─ {map_id}.webconf
├─ data/
│  ├─ Earth
│  │  ├─ Bmng
│  │  │    └─ {map_id}/{z}/{x}/{y}.idx
│  │  │    └─ {map_id}/{z}/{x}/{y}.jpg
│  │  │    └─ {map_id}/{z}/{x}/{y}.mrf
│  │  ├─ Gebco
│  │  │    └─ {map_id}/{z}/{x}/{y}.ppg
│  │  │    └─ {map_id}/{z}/{x}/{y}.png
│  │  │    └─ {map_id}/{z}/{x}/{y}.mrf
│  ├─ source_images/
│  │  └─ {map_id}.tif
├─ documents/
│  └─ {document_id}
├─ genconf.sh
├─ setup-ahtse.sh
├─ README.md
└─ .gitignore
```

Directory Summary

1. **.devcontainer/**
   Contains the Visual Studio Code development container configuration. Defines the environment, tools, and extensions required for development consistency.
2. **.github/**
   Contains repository-level configuration files used by GitHub, including:
   - **CODEOWNERS** – Defines repository ownership and review responsibilities.

- **workflows/** – Contains CI/CD pipeline definitions (for example, commit message validation)

3. **webconf/**
   Contains configuration files (.webconf) that define how specific datasets (maps or layers) are rendered within the visualization engine.
4. **data/**
   Stores all imagery and elevation datasets. Data is organized in a hierarchical, tile-based structure that supports multi-resolution map rendering.
5. **documents/**
   Contains project documentation, dataset metadata, or generated reports.
6. **Shell Scripts:**

   - **genconf.sh** – Generates configuration files for datasets.
   - **setup-ahtse.sh** – Initializes the Advanced Hierarchical Tile Service Engine (AHTSE) used to serve tiled raster data.

7. **README.md**
   Provides a project summary, setup instructions, and contributor information.
8. **.gitignore**
   Defines which files and directories are excluded from version control.

Data Directory Structure:

```
├─ data/
│  ├─ Earth
│  │  ├─ Bmng
│  │  │   └─ {map_id}/{z}/{x}/{y}.idx
│  │  │   └─ {map_id}/{z}/{x}/{y}.jpg
│  │  │   └─ {map_id}/{z}/{x}/{y}.mrf
│  │  ├─ Gebco
│  │  │   └─ {map_id}/{z}/{x}/{y}.ppg
│  │  │   └─ {map_id}/{z}/{x}/{y}.png
│  │  │   └─ {map_id}/{z}/{x}/{y}.mrf
│  ├─ source_images/
│  │  └─ {map_id}.tif
```

- The **data/** directory contains the geospatial raster data used by the visualization system. Files are stored by dataset and tile coordinate, allowing for efficient retrieval at multiple zoom levels.
- *A typical file path is formatted as follows*:

```
data/Earth/<DatasetName>/<MapID>/<Z>/<X>/<Y>.<extension>
```

- Where:
    - **MapID** – A string identifying the dataset (for example, “BlueMarble”).
    - **Z** – Integer representing the zoom level (0 = lowest resolution).
      X – Integer representing the horizontal tile coordinate.
    - **Y** – Integer representing the vertical tile coordinate.

File Descriptions

1. **Index File (.idx)**
    a. **Purpose**: Holds metadata about tile positions and sizes within the associated Meta
2. **Raster Format (MRF) data file**
    a. **Purpose**: Provides structural and metadata information describing a multi-resolution raster dataset.
3. **Image Tiles (.jpg, .png, .ppg)**
    a. **Purpose**: Contain the actual raster image data used for display at specific coordinates and zoom levels.
4. **Source Imagery (.tif)**
    a. **Purpose:** High-resolution GeoTIFF source images are used to generate the tiled datasets.
5. **Web Configuration File (.webconf)**
    a. **Purpose:** Defines how datasets are displayed in the web visualization interface, including paths, zoom parameters, and projection details.
    b. **Key Fields**
        i. RegExp (string) = "*./tile/.*" - URL/path pattern that this configuration applies to.
        ii. Size (int int int int) = "4978 6322 1 3" - Raster width, height, bands, and data-type code.
        iii. PageSize (int int int int) = "512 512 1 3" - Tile width, tile height, bands, and data-type code.
        iv. DataFile (string, path) = "/<project-path>/data/Earth/Bmng/test.pjg" - Backing raster data file.
        v. IndexFile (string, path) = "/<project-path>/data/Earth/Bmng/test.idx" - Index file mapping tiles within DataFile.
        vi. SkippedLevels (int) = 0 - Number of pyramid levels omitted from serving.
        vii. Example webconf:

```
RegExp .*/tile/.*
Size 4978 6322 1 3
PageSize 512 512 1 3
DataFile /<project-path>/data/Earth/Bmng/test.pjg
IndexFile /<project-path>/data/Earth/Bmng/test.idx
SkippedLevels 0
```

6. **Development Container Configuration (devcontainer.json)**
    a. **Purpose:** Defines the local development environment used by the team.

b. **Key Fields:**
   i. name (string) = "Ubuntu" - Human-readable environment name.
   ii. image (string) = "mcr.microsoft.com/devcontainers/base:noble" - Base container image.
   iii. features (object)
   iv. ghcr.io/devcontainers/features/git-lfs:1 (object) - Enables Git LFS.
   v. ghcr.io/devcontainers-extra/features/apt-packages:1 (object)
   vi. packages (string) = "gdal-bin,libgdal-dev,build-essential,cmake,apache2,apache2-dev,libapr1-dev,libaprutil1-dev,libjpeg-dev,libpng-dev,zlib1g-dev,libcurl4-openssl-dev,sqlite3,libsqlite3-dev,git" - Apt packages to install.
   vii. postCreateCommand (string) = "bash setup-ahtse.sh && bash genconf.sh" - Command run after container is created.

7. **GitHub Workflow Configuration (validate-commit-message.yml)**
   a. **Purpose:** Defines automated actions used by GitHub to validate commit messages or run tests.
   b. **Key Fields:**
      i. Name: "Validate Commit Message" — Workflow name.
      ii. On: push — Event trigger.
      iii. Runs-on: "ubuntu-latest" — Operating system for the runner.
      iv. Uses: "actions/checkout@v4" — Checks out the repository.
      v. Fetch-depth: 0 — Ensures full commit history is available.
      vi. Id: "range" — Identifier for the step calculating the commit range.
      vii. Shell: bash — Execution shell for steps.
      viii. Run (range): Script that determines the “from” and “to” commit SHAs.
      ix. Name (validation): "Validate commit types" — Step that checks commit message format.
      x. VALID_TYPES: "feat|fix|chore|refactor|test|docs|ci" — Allowed commit prefixes.
      xi. Run (validation): Bash loop that validates each commit message against the allowed format.

8. **genconf.sh**
   a. **Purpose:** Scans ./webconf for .webconf files, validates that corresponding data/index files exist under the data root, fixes DataFile/IndexFile paths in-place, and generates ahtse.conf with Apache/AHTSE directory blocks for serving MRF tiles.
   b. **Key Fields:**
      i. DATA_FOLDER: /workspaces/OpenSpace-x-ClimateReanalyzer/data — Root directory for MRF data and tiles (can be overridden by environment variable).
      ii. WEBCONF_ROOT: <repo-root>/webconf — Directory containing

.webconf configuration files.

iii. OUT_CONF: <repo-root>/ahtse.conf — Output file generated for Apache/AHTSE configuration.

iv. Required Lines: "DataFile <path>" and "IndexFile <path>" — Must be present in each .webconf file.

v. Supported Extensions: DataFile uses .ppg or .pjg; IndexFile uses .idx.

vi. Swap Detection: Automatically corrects swapped file references (e.g., .idx listed as DataFile).

vii. Fallback Search: Checks alternate directories if expected file paths are missing.

viii. Path Normalization: Rewrites .webconf entries with verified absolute file paths.

ix. Apache Block Output: Adds configuration for each dataset to ahtse.conf with:

1. Options: -Indexes -FollowSymLinks -ExecCGI
2. MRF_RegExp: */tile/.*
3. MRF_ConfigurationFile: Absolute path to the corresponding .webconf.

x. Logging and Safety: Uses strict error handling (set -euo pipefail) and detailed logging.

xi. Dependencies: Requires bash, sed, awk, grep, find, and realpath/readlink.

9. **setup-ahtse.sh**

a. **Purpose:** Automates installation of system dependencies; builds AHTSE-related libraries and Apache modules from source; registers/load-files for Apache; creates a site config exposing the project's data and including the generated ahtse.conf.

b. **Key Fields:**

i. Package Installation: Installs core dependencies, including Apache, GDAL, build tools, and development libraries.

ii. Apache Paths:

1. APXS_BIN: apxs — Apache extension tool.
2. APR_INC: APR includes the directory path.
3. AP_INC: Apache include directory path.
4. AP_LIBEXECDIR: Apache module directory path.

iii. Include Paths: Adds Apache and APR include directories to environment variables (CPATH and CPLUS_INCLUDE_PATH).

iv. Build Directories: Creates required folders under the user home directory — wms_modules, modules, lib, and include.

v. Dynamic Linker Config: Registers $HOME/lib in /etc/ld.so.conf.d/ahtse.conf and runs ldconfig.

vi. Repositories Cloned:

vii. libahtse, AHTSE, libicd, mod_mrf, mod_receive, mod_sfim, mod_reproject,

mod_convert (from GitHub).

viii. Build Order: Builds and installs modules in dependency order:

ix. mod_receive → libicd → libahtse → mod_mrf → mod_convert → mod_reproject → mod_sfim.

x. Build Configuration: Generates local Makefile.lcl files with proper include paths; uses CMake as a fallback for libicd.

xi. Installed Files:
    1. Compiled modules stored in $HOME/modules.
    2. Headers installed to $HOME/include.
    3. Some modules are also installed in Apache's module directory.

xii. Apache Module Load Files: Creates and enables .load files in /etc/apache2/mods-available for available modules (mrf, convert, receive, retile, sfim).

xiii. Site Configuration:
    1. Creates /etc/apache2/sites-available/100-ahtse.conf with:
    2. Alias /tiles/ mapped to /workspaces/OpenSpace-x-ClimateReanalyzer/data/

xiv. Includes the generated ahtse.conf file.

xv. Validation: Runs apachectl configtest and restarts Apache. Verifies loaded modules using apachectl -M.

Summary:

The OpenSpace-x-ClimateReanalyzer system relies on structured and standardized file formats to ensure interoperability across datasets, visualization components, and processing tools.
Each file type described here contains sufficient metadata and data fields for a developer to implement loading, parsing, and serving mechanisms within the system.

4 Requirements Matrix

The following table identifies which system components satisfy each of the functional requirements for the OpenSpace and Climate Reanalyzer Interface. The functional requirements are defined in more detail in the System Requirements Specification document.

Requirement Use Case # / Name	Use Case Description	System Component(s)

1: Map - Tile Conversion	Convert 2:1 map image into tiled map format and display in OpenSpace.	check_availability(map_id) download_image(CR_image) generate_tiles(image) send_tiles(tiles)
2: Download and Save Map Files	Download and locally store selected tile maps for offline use.	select_image(image) generate_tiles(image) download_tiles(tiles, path)
3: Admins Uploading Large Data Files	Upload source maps to the tile server	upload_map(map_id, path)
4: List Available CR Maps	List available Climate Reanalyzer maps and metadata	check_maps() get_matadata(maps) send_metadata(metadata)
5: Multiple tile levels	Provide inner layers of tiles and display in OpenSpace	generate_tiles(image, layer) send_tiles(tiles)
6: Animating Maps and Data	Display animation of data over a chosen time range for selected region	get_maps(map_class, time_range) generate_tiles(image) send_tiles(tiles)
7: Handling Errors and Status Feedback	Alert users upon system errors	catch(error) log(error) display(error)

Appendix C – User Interface Design Document (UIDD)

Table of Contents

	Page
Appendix C – User Interface Design Document (UIDD)	**36**
1. Introduction	36
1.1 Purpose of This Document	37
1.2 References	37
2. User Interface Standards	38
3. User Interface Walkthrough	40
3.1 Navigation Diagram	40
3.2 Screen Descriptions	41
4. Data Validation	43
4.1 Data Items	43

1. Introduction

This project, titled OpenSpace and Climate Reanalyzer Interface, is a comprehensive capstone project for University of Maine Computer Science students on V-Model Violets (formerly known as Team Violet). This project will be developed in collaboration with Dr. Sean Birkel, a Research Assistant Professor at the Climate Change Institute, and Sean Laatsch, the Director of the Versant Planetarium at the University of Maine. This project will also be collaborating with Dr. Carter Emmart and Micah Acinapura from the OpenSpace team. The goal of this project is to create an image map server that utilizes data from the Climate Reanalyzer, enabling its use by OpenSpace and in planetariums or other research settings.

1.1 Purpose of This Document

The purpose of this User Interface Design Document (UIDD) is to detail the standards, design, and data validation for the user interface of the OpenSpace and Climate Reanalyzer Interface. The document also describes how the user will interact with the interface to access all of the product's functionalities. This document will serve as a blueprint for our team, the clients, and the teaching staff for the Capstone class, outlining the functionality and appearance of the user interface. The UIDD is intended for:

- Capstone Team Members: To guide the group through the implementation process, as well as ensure the client's needs are met.
- Clients: To show how we plan to construct the system in a transparent and informative way.
- University of Maine Faculty: To assess the progress of the project.

1.2 References

The table below lists the sources being used for this project currently:

Title	Author(s)	Publisher / Organization	Date	Citation
Climate Reanalyzer	Dr. Sean Birkel	Climate Change Institute, University of Maine	Ongoing	Birkel, S. (n.d.). Weather forecasts. Climate Reanalyzer. https://climatereanalyzer.org/
OpenSpace	OpenSpace Team	American Museum of Natural History	Ongoing	OpenSpace. (n.d.). OpenSpace. https://www.openspaceproject.com
Apache HTTPD Server	Apache Software	Open Source	1997-Current	Apache Software Foundation. (n.d.). Apache

Documentation	Foundation			HTTP Server Project. https://httpd.apache.org/
Software Engineering (10th Edition)	Ian Sommerville	Pearson Education	2015	Sommerville, I. (2015). Software engineering (10th ed.). Pearson Education.

2. User Interface Standards

The following standards outline our goals for the appearance and functionality of our user interface regarding components, navigation, error handling, and overall design.

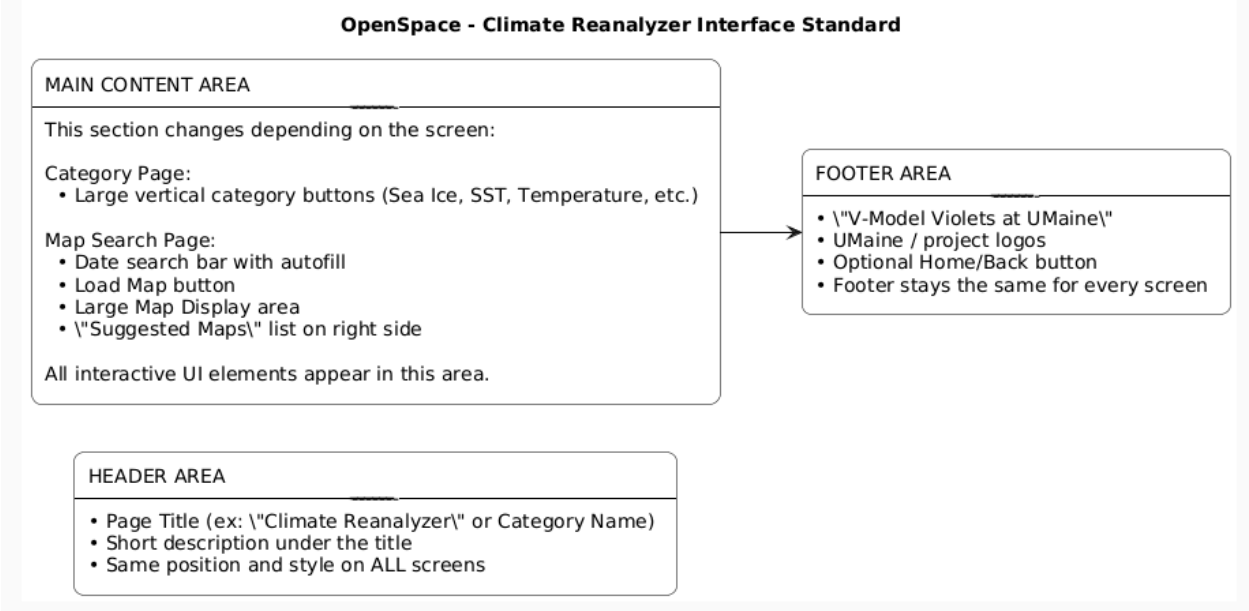

Figure 2.1 OpenSpace - Climate Reanalyzer User Interface Standard

Common Components.
Button Style: Rounded rectangle, consistent size, same colors across screens
Search Bar: autofill with consistent placement.
Category buttons: Same layout and spacing on all category pages
Suggested Maps Panel: will appear on every Map Search Page
Header title
Back/Home Navigation: Same design across all screens

Navigation Standards:
Users will always return to the Landing Page using the back button in the header
The elements of the navigation appear in consistent locations. Footer at the bottom, back button top left of screen
Users navigate specific map pages using vertically stacked buttons
Horizontal scrolling. The navigation will flow vertically.

Design Principles:
Minimalist layout is used across all screens
Light text on dark backgrounds. The text will be high contrast.
Consistent font styles for headed and body text.
Buttons use consistent highlight color for interactivity.
Icons follow a unified visual style.
Important elements (search bar) will use a visual emphasis (larger size.)

Error Handling
Error messages will appear directly under the search bar if date is invalid
Warning messages will use a consistent color
Errors will not black the layout. The will appear inline so users can correct inputs easily.
Load map button remains disabled until the input is valid.

The interface will follow a consistent layout throughout all screens. The three-part layout will have a header area at the top. A main content area in the center and a footer area at the bottom. The layout is predictable, making it easier for users to navigate and find important information that will be located in the same locations across all screens. The header will include the screen title and a description, which helps users understand the context of the page and the information they can expect to see and interact with. Figure 2.1 in the “MAIN CONTENT AREA”. It says “This section changes depending on the screen.” This means that the main content area changes depending on which page the user is on. The interface design of the main content area will remain consistent.

The main content area changes depending on the user’s workflow within the system. For example, on the Category Page, the main area displays large buttons that go vertically down the main section. These buttons are categorized, such as Sea ice, Sea Surface Temperature (SST), or

temperature. On the Map search page, the main area includes the date search bar, the load map button, the map display region, and a sidebar containing suggested maps. The content varies, but the area consistently holds all of the interactive elements that the user uses. Structuring the system this way makes sure the users always know where to look on each screen.
The footer will provide information, including "V-Model Violets at Umaine", branding Umaine logos, and an optional navigation link, such as home or back, if needed. The footer will remain the same on every page. This will help reinforce project identity. This three-part layout header, main content, and footer form the interface standard that all future screens in the system will follow. In addition to this three-part layout, the interface will use common components, navigation rules, design principles, and error handling to ensure consistency across all screens.

3. User Interface Walkthrough

The OpenSpace and Climate Reanalyzer Interface's User Interface will be embedded within the OpenSpace application. Below is a walkthrough of the functions the user will have access to via the User Interface, and how the product will appear as two separate page types that the user will be able to navigate between.

3.1 Navigation Diagram

The Navigation Diagram shown in Figure 3.1 shows the two different types of pages that will be implemented in our user interface, and how the user will navigate between them. Upon opening the Climate Reanalyzer x OpenSpace Interface within the OpenSpace application, the user will be directed to the Landing Page. From there, the user will see multiple buttons that each bring the user to a corresponding Map Search Page. The user may click the back button on the top left to return to the Landing Page.

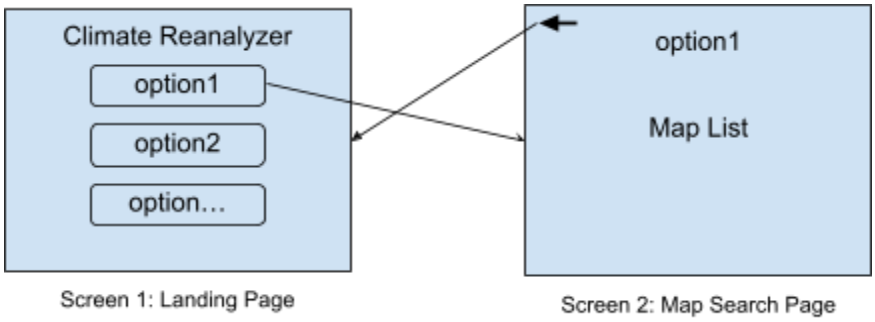

Figure 3.1: Navigation Diagram for Climate Reanalyzer x OpenSpace Interface

3.2 Screen Descriptions

The user interface will consist of two different screen types as shown below.

Screen 1: Landing Page

The first screen will be the landing page, which appears when the user first chooses to view the Climate Reanalyzer Maps through OpenSpace. The window will be opened in OpenSpace itself and will generally be rectangular in size with it being taller than it is wide. It will feature the title: "Climate Reanalyzer" at the top with a short description: "Various Global Maps Collected by the Climate Reanalyzer at the University of Maine." It will also feature buttons with the map categories. The user will be able to choose which category they would like to view the maps of. When a category is selected, the system will navigate to a new page, Screen 2 from the Navigation diagram, displaying the maps with available dates for the user to pick from. It will also feature the V-Model Violets and the University of Maine Logos with the text "V-Model Violets at UMaine" at the bottom of the screen.

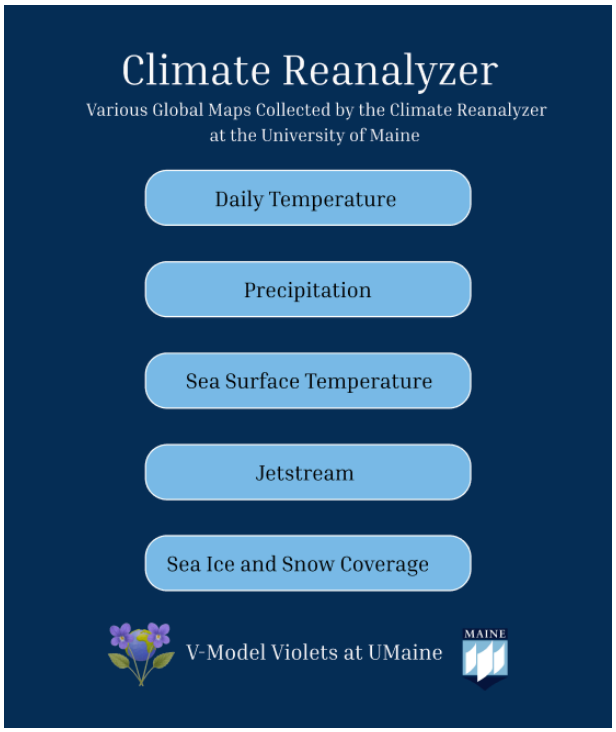

Figure 3.2.1: Screen 1: Landing Page

Screen 2: Map Search Page

The second screen will be the page you are directed to after making your selection on the landing page. The second screen will have different variations depending on which selection was made, but functionally, they are the same. The screen will include a title that matches the selection from screen 1, followed by a short description of the chosen option.

Underneath the description will be a calendar picker that will prompt the user to enter the desired date for the map they would like to view. The user will then either manually enter their desired date in the bar following the placeholder format, or may click the calendar icon to select a date from a calendar view.

Near the bottom of the page will also be some hand-picked interesting maps that users can select by clicking the underlined date. These options will be hard-coded in as links and will vary depending on the selection made on screen 1.

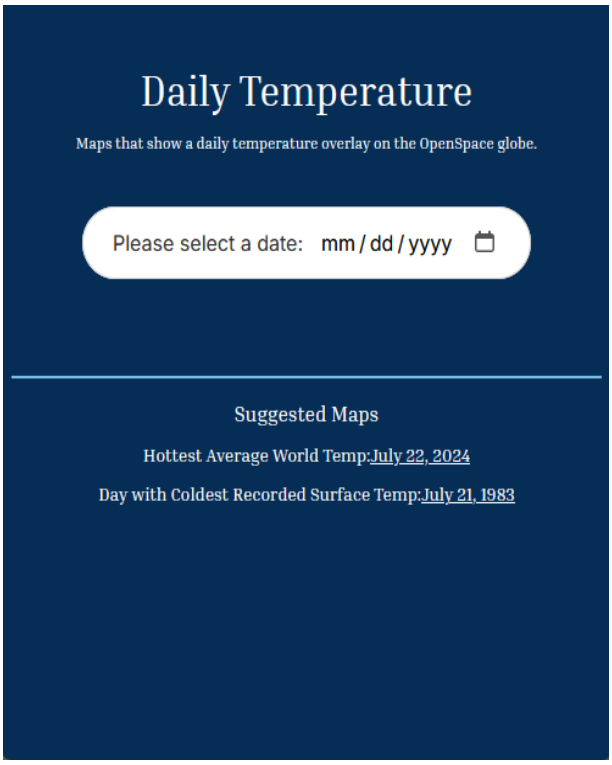

Figure 3.2.2: Screen 2: Map Search Page

4. Data Validation

The OpenSpace and Climate Reanalyzer Interface accepts a small range of inputs from the user that must be properly validated to allow the user to accomplish the desired actions. This section describes the inputs, including the data types, acceptable formats, and validation methods. The list will be provided below in tabular form.

4.1 Data Items

Screen	Data Item Label	Data Type	Allowed Values	Format / Example	Validation Rules
Landing Page	Category Selection Button	String (Enumeration)	“Daily Temperature”, “Precipitation”, “Sea Surface Temperature”, “Jetstream”, “Sea Ice and Snow Coverage” (there may be more depending on data)	One Selection Only	Selection required; the user must choose a valid category button before proceeding.
Map Search Page	Date Input	String (Date)	Valid dates from January 1, 1979, through the current date	MM-DD-YYYY (e.g., 01-12-1997)	Input must match MM-DD-YYYY format; invalid or future dates trigger a warning
Map Search Page	Suggested Map Link	String (URL)	Predefined links to existing datasets	https://climatereanalyzer.org/maps/...	Validation will not be needed here, as they will be given, and the user cannot change them